LifeBox NextGen

Sprint 1 Detailed Execution Plan

Sprint Duration: March 2, 2026 to March 7, 2026

Sprint Objective: Build the Foundation of the Entire System

Sprint 1 Core Purpose

The main purpose of Sprint-1 is to establish the **technical foundation on which the entire LifeBox NextGen system will run**.

This includes:

- Backend server creation
- Database setup and connection
- Desktop application base creation
- Login system implementation
- Monitoring proof-of-concept (detecting active application and sending it to backend)

This sprint is critical because every future module—attendance, screenshots, dashboards, payroll—depends on this foundation.

For example:

If login is not working properly, employees cannot enter the system.

If monitoring data cannot reach the backend, your entire monitoring system becomes useless.

March 2, 2026 – Backend Project Setup and Database Initialization

Backend Development Tasks – Project Initialization

On this day, the backend developer must create the main backend server project using a structured framework such as NestJS or Express.js.

The purpose of this backend server is to act as the central system that receives data from desktop applications and provides data to admin dashboards.

The developer must create the initial project and verify that the server runs successfully.

Expected outcome:

When the developer runs the command to start the server, the backend should start successfully and be accessible at a URL such as:

http://localhost:3000

This confirms that the backend server is operational.

Database Setup and Connection

On the same day, PostgreSQL must be installed and configured.

A new database must be created with the name:

lifebox_nextgen

This database will store all system information, including users, attendance, monitoring logs, and tasks.

After creating the database, the backend must be connected to this database using proper database connection configuration.

Expected outcome:

The backend server must successfully connect to the database without errors.

This can be verified if the backend starts without showing database connection errors.

Desktop Application Initialization

The desktop developer must create the initial desktop application using Electron.js.

The purpose of this application is to act as the employee's working interface and monitoring tool.

The developer must ensure the application launches successfully.

Expected outcome:

When the developer runs the Electron project, a desktop window should open.

Even if it shows a blank screen, this confirms the desktop application is ready for further development.

March 3, 2026 – Login API and Desktop Login Interface

Backend Login API Development

On this day, the backend developer must create the login API.

The login API is responsible for verifying employee credentials and allowing employees to enter the system.

This API will receive:

- Employee email
- Employee password

The backend must check these credentials against the database.

If the credentials are correct, the backend must allow login.

Expected outcome:

When login credentials are sent using tools like Postman, the backend must return a success response.

This confirms login functionality is working.

Insert Test Employee in Database

To test login functionality, at least one employee record must be manually added to the database.

For example:

Email: admin@lifebox.com
Password: (stored as encrypted password)

This test account will be used to verify login.

Expected outcome:

Using this test account, login must succeed.

Desktop Login Screen Development

The desktop developer must create the login interface.

This interface must include:

- Email input field
- Password input field
- Login button

This allows employees to enter their credentials.

Expected outcome:

The desktop application must display the login screen and allow the user to type credentials.

March 4, 2026 – Connecting Desktop Login with Backend Authentication

Backend JWT Authentication Implementation

The backend developer must implement JWT authentication.

JWT is required because it allows the backend to securely identify users after login.

After successful login, the backend must generate a token and send it to the desktop application.

This token will be used for all future communication.

Expected outcome:

When login is successful, the backend returns a token.

Desktop Application Login Integration

The desktop developer must connect the login screen to the backend login API.

This means when the employee clicks login, the desktop application sends credentials to the backend.

If login is successful, the desktop application must receive the token.

Expected outcome:

Employee enters credentials in desktop application and successfully logs in.

This confirms communication between desktop and backend works.

March 5, 2026 – Monitoring System Initial Development

This is the first day of monitoring system implementation.

Monitoring is the most critical part of your LifeBox system.

Desktop Application Monitoring Script

The desktop developer must create a script that detects which application is currently active.

This means the system must identify whether the employee is using:

Chrome
VS Code
File Explorer

The system must capture the active application name.

Expected outcome:

The desktop application must detect and display the active application name.

For example:

If employee opens Chrome, system detects Chrome.

Backend Monitoring API Development

The backend developer must create an API endpoint that receives monitoring data.

This API will accept:

Employee ID
Application name
Time

Expected outcome:

Backend must receive monitoring data successfully.

March 6, 2026 – Sending Monitoring Data to Backend and Saving in Database

Desktop Application Monitoring Integration

The desktop developer must modify the monitoring script to send detected application data to the backend automatically.

This must happen at regular intervals, such as every 30 seconds.

Expected outcome:

Desktop application automatically sends application usage data.

Backend Database Storage Implementation

The backend developer must create a database table called application_logs.

This table stores:

Employee ID
Application name
Time

The backend must save monitoring data into this table.

Expected outcome:

Monitoring data must be visible in the database.

For example:

Database shows employee used Chrome at 10:05 AM.

March 7, 2026 – Full Integration Testing and Sprint Validation

This is the most important day of the sprint.

All developed components must be tested together.

The complete system flow must work as follows:

Employee opens desktop application
Employee logs in successfully
Desktop application detects active application
Desktop application sends monitoring data to backend
Backend receives data
Backend saves data in database

Expected outcome:

When checking the database, monitoring data must be visible.

This confirms monitoring pipeline works.

Sprint 1 Final Deliverable Summary

By the end of Sprint-1, the following must exist and work properly:

Backend server running successfully

Database connected and storing data

Desktop application launching successfully

Employee login working from desktop

Desktop detecting active application

Desktop sending monitoring data to backend

Backend saving monitoring data in database

Why This Sprint is Critical

This sprint builds the system backbone.

Without this:

Attendance cannot work

Screenshot monitoring cannot work

Admin dashboard cannot work

Everything depends on this foundation.

For example:

If monitoring pipeline fails now, you cannot fix everything in last week.